# AutoGT: Automatic Generation of Game Trees for Algorithm Benchmarking

Jan Karwowski, Maciej Świechowski, and Jacek Mańdziuk, *Senior Member, IEEE*

***Abstract*—Game tree search algorithms play a vital role in application of computational and artificial intelligence methods in games and sequential combinatorial optimization problems. This article addresses the need for robust and comprehensive benchmarking of these algorithms through the procedural generation of benchmarks. We introduce a novel Automatic Game Tree automatic game tree (AutoGT) generator built upon the idea of top–down propagation of various game parameters. Thanks to its extensive parameterization, AutoGT enables researchers to evaluate their methods on virtually unlimited number of unique benchmarks. To demonstrate the efficacy and usefulness of the generator, we present comparative results of several standard game-playing algorithms: a game theory optimal player, a player making moves according to a uniform random distribution, and a few variants of Monte Carlo Tree Search-based players. Our findings indicate that for a given set of benchmark parameters, the results are repeatable with low deviation. However, when testing across various parameter settings, the performance becomes more diverse, allowing for a broader and more detailed assessment of the algorithms. This article includes a link to the full code of AutoGT benchmark generation method.**

***Index Terms*—Game theory, game tree search, Monte Carlo tree search (MCTS), procedural generation.**

## I. Introduction

GAMES have been widely recognized as a promising benchmark for artificial intelligence (AI) methods since the early seminal contributions, such as the Min–Max theorem by Neumann and Morgenstern [1], Alan Turing's work on chess [2], or the first checkers program by Samuel [3]. On a general note, games serve as cheap, controllable, repeatable testing frameworks with formally defined actions and reward structures, what makes them an ideal testbed for the development of new algorithms and their comprehensive comparisons [4].

The importance of games as a testing framework continued with chess, named the "drosophila of AI" [5], whereas *Go* was labeled as "the Grand Challenge of AI" [6]. Nowadays, games are often utilized to benchmark new AI/machine learning (ML) techniques, e.g., deep reinforcement learning variants in Atari games [7], *Starcraft II* [8], or *Dota 2* [9].

In general, there are many diverse types of approaches that can be benchmarked using combinatorial games that refer to the following game-playing or game-solving techniques.

1) Search and planning algorithms.
2) Combinatorial optimization methods.
3) Metaheuristics, including evolutionary approaches (EA).
4) ML, including modeling state evaluation (value) and policy functions.

At the same time, a few common problems and limitations can be pointed out. First, many research papers focus on one specific game, and consequently, their primary outcome usually concerns the progress related to that particular game, such as new game-related insights or a new best-performing approach. However, significant uncertainty remains with respect to how the proposed approach would perform across a diverse pool of games or other decision-making problems. Second, when a new state-of-the-art approach is proposed, there is typically no objective measure of its absolute effectiveness unless it solves the game. Until the game is solved, which has been possible only for games with relatively low complexity, the optimal strategy remains unknown.

Motivated by the above-mentioned concerns, our main contribution in this article is three-fold:

1) We propose a procedural way of generating parameterized and diverse game models in the form of game trees of variable length. The notion of a game tree will be described in more detail in Section III. Each game tree represents a generated game instance that can be used to test custom tree search methods or game-playing approaches. Importantly, and contrary to other approaches to game tree generation presented in the literature, each game tree is designed with a game-theoretically optimal (GTO) score determined by Nash equilibrium [10]. This score should not be provided to the tested algorithms but can be used for various objective evaluations of their efficacy.
2) We demonstrate the potential of automatic game tree (AutoGT) by comparing several variants of Monte Carlo Tree Search (MCTS) agents [11], [12]—one of the state-of-the-art techniques of searching game trees.
3) We release the code of the AutoGT generator with an open license to the game community to encourage further research developments in the area of automatic benchmark generation.

Received 28 September 2024; revised 4 February 2025, 27 March 2025, and 25 April 2025; accepted 29 April 2025. Date of publication 5 May 2025; date of current version 17 December 2025. Recommended by Associate Editor A. Dockhorn. *(Corresponding author: Maciej Świechowski.)*

Jan Karwowski is with the Faculty of Mathematics and Information Science, Warsaw University of Technology, Koszykowa 75, 00-662 Warsaw, Poland.

Maciej Świechowski is with QED Games, Warsaw University, Miedziana 3A, 00-814 Warsaw, Poland (e-mail: maciej.swiechowski@qed.games).

Jacek Mańdziuk is with the Faculty of Mathematics and Information Science, Warsaw University of Technology, Koszykowa 75, 00-662 Warsaw, Poland, and also with the Faculty of Computer Science, AGH University of Krakow, Adama Mickiewicza 30, 30-059 Krakow, Poland.

Digital Object Identifier 10.1109/TG.2025.3566891

In this study, we focus exclusively on two-player, deterministic games, which is a deliberate choice aimed at establishing a clear and manageable foundation for the proposed game tree generation algorithm. In addition, such games are the most popular in the realm of classic combinatorial games, e.g., *Go* and *Chess*. Generalization to an arbitrary number of players is relatively straightforward. However, incorporating nondeterminism will necessitate further research, in particular defining what constitutes an optimal move. The exploration of relaxation of these two constraints is left for subsequent studies.

The rest of this article is organized as follows. Section II provides more background and discusses the related literature. Section III introduces the method—the tree generation algorithm. Section IV describes the experimental setup, including the playing agents used in the experimental comparison that illustrates the usage of AutoGT, while Section V presents the empirical results. Finally, Section VI concludes this article.

## II. Related Work

In the context of generating game models, it is worth mentioning multigame playing approaches, as they involve testing search/prediction/planning techniques on a diverse set of games. Pell's [13] METAGAMER was one of the first programs capable of playing a variety of games without any fine-tuning for each of them. It included a generator for symmetric chess-like games, i.e., games in which two players could move pieces of various kinds on a rectangular board. The goals of the games involved eliminating certain types of pieces, making the opponent run out of available moves, or transferring specific pieces to designated coordinates on the board.

The general game playing (GGP) [14], [15] was proposed by the Stanford Logic Group as a research framework for multigame playing. An annual GGP Competition was organized between 2005 and 2016. The game benchmarks were defined by humans using a logic-based language called game description language (GDL). These benchmarks were either GDL adaptations of existing games, such as *chess*, *checkers*, or *Othello*, or were designed solely for the purpose of the competition.

The general video game AI competition [16] offered a similar framework tailored for simple real-time video games, akin to those found on early arcade or Atari systems. Herein, the benchmarks were also prepared by humans—without the use of procedural generation—in the so-called Video Game Description Language (VGDL). The top-performing programs were typically based on the Rolling Horizon Evolutionary Algorithm [17] or MCTS.

In [18], an attempt to automatically generate games represented in VGDL using evolutionary algorithms is reported. The authors introduced a fitness function based on how well it differentiates between stronger and weaker game-playing algorithms. The attempt is assessed as "partially successful" due to the inability to pinpoint interesting games, frequent requirement of superhuman response times to play them, and computational complexity, with the evolution process taking several days.

Evolutionary algorithms were also employed by Browne and Maire [19] to generate combinatorial board games represented in the so-called "ludemic" description, which uses LISP-like terms, such as *[board (tiling square) (size 3 3)]*. The authors used various metrics, empirically measured through simulations, to assess the evolved games. This process was time-consuming, as the goal was to generate games that were meaningful for humans. One game, called *Yavalath*, turned out to be very popular and was published commercially.

The automatic generation of game rules, represented as expression trees using the zillions of games game description language, was explored in [20].

Another example is variations forever [21] that combines a formal logic-based language with a specified vocabulary of possible keywords to generate games. Herein, the authors utilize answer-set programming rather than an EA.

The authors in [22] and [23] presented comprehensive surveys on procedural content generation (PCG) and propose various taxonomies. While these taxonomies primarily focus on game content, certain features can be attributed to our generator, e.g., "system design (Abstract)" and "pseudorandom number generators (PRNGs)" according to the taxonomy from [22], or "online," "constructive," and "random seeds" according to the taxonomy keywords from [23]. Specifically, Togelius et al. [23] focused on search-based PCG approaches for games, where the authors note an overrepresentation of evolutionary methods.

In contrast to all mentioned works, our generator: 1) is extremely fast, 2) focuses on abstract game trees (rather than human-playable games), thus allowing for comprehensive comparisons of various algorithms, and 3) generates, by design, optimal scores in each node of the tree. These scores can be used for objective evaluation of the algorithms' performance as well as testing various heuristics, within the same algorithm, as shown in Section IV-B.

The underlying idea of this article is not limited to the domain of games but also extends to planning problems. For instance, The International Planning Competition (IPC) [24], hosted at the ICAPS conference, features five tracks representing different types of planning problems. Procedural generators could enhance this competition by dynamically producing diverse problem instances that vary in size, complexity, and difficulty. Moreover, the "optimal" subtrack within one of the IPC's tracks could particularly benefit from the AutoGT generation method, due to its inherent (by design) access to optimal scores.

In recent studies [25], [26], various upper confidence bounds applied to tree (UCT)-like variants for MCTS are tested on artificially constructed game benchmarks. UCT [27] is a specific selection policy. Each game is represented as a 1-D function optimization problem. The authors define five functions, e.g., the one as follows:

$$f_2(x) = 0.5\sin(13x)\sin(27x) + 0.5. \tag{1}$$

The goal of the game is to find the global optimum of the function. Each state is associated with an interval, e.g., the root state typically corresponds to [0,1].

The branching factor $b$ is a parameter of the generator. Let $[x, y]$ denote an interval in the current state. Each action available to the player narrows down the interval of the respective child

state to $[x + \frac{(i-1)(y-x)}{b}, x + \frac{i(y-x)}{b}]$, where $i$ is the action's index. This research is similar to ours in the aspect that game instances are artificially created and serve the purpose of testing game-tree search algorithms. However, the game structures in [25] and [26] are very limited in terms of rewards, game tree shapes, and actions, especially due to the uniform intervals' division. Moreover, their experimental evaluation is restricted to various UCT variants only.

Comparative research on game tree search algorithms and other game-playing techniques is significantly more popular than the automatic generation of benchmarks. Most research papers that demonstrate the use of MCTS in a given game include comparisons of several variants or parameterizations. Enumerating all of them in specific games (and nongame problems) is, therefore, infeasible. For example, Mańdziuk and Walczak [28] compared the efficacy of several MCTS variants enhanced with metaheuristics with alpha–beta on *Othello and Hex*. The authors in [29] and [30] included MCTS modifications across several games from the general video game playing (GVGP) corpus. One conclusion from these studies, aligning with the "no free lunch theorem," is that certain modifications are suitable for specific classes of games. Another finding is that certain modifications can be combined, resulting in an overall stronger player compared to using just one of them. The fact that no single variant dominates the others is also confirmed in our studies [31], [32] concerning the GGP. In [31], various ways of incorporating the evaluation function into MCTS are considered. For example, it can be used as part of the Q value evaluation during MCTS tree construction, for move ordering during the search, to guide random rollouts (simulations), or to dynamically terminate rollouts at some level and return the heuristic evaluation instead. In [32] and [33], the GGP-related approach, in the form of a specific policy-interleaving enhancement of the MCTS/UCT simulation phase, is proposed and experimentally evaluated. For more information about various modifications of MCTS and their performance in specific problems, please consult the survey papers [11], [12].

The overall popularity of MCTS, including its use in the seminal *AlphaGo* [34] program, convinced us to consider it also in our research. In the experiments, we employ a few of its most popular variants, as our goal is not to propose new MCTS modifications, but rather introduce a novel benchmarking method and demonstrate its efficacy, by means of an experimental comparison of selected MCTS variants.

In the context of the above-discussed related work, there are several research paths, beyond the scope of this article, for which application of AutoGT could be feasible. We summarize these potential applications in the next two paragraphs and leave their experimental verification for future work.

First, AutoGT can also be used with other tree-search algorithms, e.g., in scenarios aimed at testing various parameterizations across many test instances or specific subsets of games defined based on the provided generator parameters. AutoGT can, for instance, be used to validate and compare alpha–beta pruning with its alternatives [35], such as negascout or MTD(f), as long as they do not require more sophisticated heuristics than those derived from AutoGT's generic parameters (optimal value with some noise/stochastic error, branching factor, depth, etc.). It could also be used to find correlations between the above parameters and the optimal settings of the algorithms' parameters, such as window sizes in alpha–beta search.

Another idea is to test variants of A$*$ algorithm, as it is possible to construct a distance function based on the number of leaves in subtrees for each move. Finally, AutoGT can serve as a test suite for implementations of game solvers [36] since early detection of their validity might save a lot of time and energy.

### A. PCG Benchmarking Outside the Game Domain

Outside the game domain, there is a growing need for benchmarks to compare various large language models (LLMs). Some of these benchmarks are generated procedurally, while others are handcrafted by experts to evaluate LLMs on specific tasks [37]. For an overview of this topic, consult a recent survey [38].

PCG is a useful approach to benchmark various reinforcement learning methods and find the best performing hyperparameters [39]. Ansotegui and Torres [40] proposed a custom algorithmic approach for generating benchmarks tailored for combinatorial testing. In [41], a tool called *Autoscale* is presented, which selects a set of instances forming the most appropriate (i.e., conforming to certain principles such as avoiding bias) benchmarks for given scenarios. Many procedural benchmark generators were proposed in the field of Dynamic Optimization. Typically, they were based either on function compositions or heuristics [42]. Finally, Yazdani et al. [43] introduced —-a generalized benchmark generator for continuous optimization.

## III. Proposed Method

We propose a parameterizable game tree generation algorithm AutoGT for two-player deterministic games. The generated trees can be used, for instance, to evaluate various tree search methods.

Each tree search method builds and represents its searched trees in its own specific way, depending on the method's implementation. Our generator does not alter this tree-building technique, and instead, in each node offers a mapping to the generator state that contains all the information required to simulate playing a game (see Fig. 1).

AutoGT maintains a vector of parameters, both global and local. Global parameters are constant within the entire game tree, whereas the values of the local parameters are node-specific.

Our generator allows for generating arbitrarily large trees, which is achieved thanks to a combination of the following properties.

1) The algorithm starts from the root and constructs the game tree in a top–down fashion. In particular, it does not need to visit the leaf nodes to compute and propagate the min–max values of the game.
2) There is a deterministic procedure that defines how local parameters associated with a given (parent) node affect the values of local parameters in successor (child) nodes. This includes the min–max values propagation.

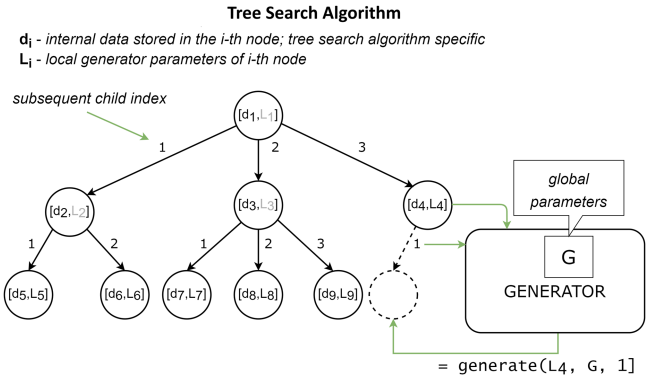


Fig. 1. Tree search algorithm maintains its own representation of a game tree. Whenever it visits (expands) a new node, such as the 10th node (dashed line circle) in this example, it passes a query to the generator providing it with the local generator parameters. The generator returns the vector of parameters that are associated with the new node. Depending on a particular search method, the fully expanded nodes, i.e., $d_1, d_2, d_3$ in the example, may no longer need to store the local generator state.

3) Parameters in a given node depend only on parameters of its parent node (the Markovian property). Therefore, the tree does not need to be stored in memory for the generation purpose.

### A. Global Parameters

1) $V^*$—Min–max game reward (in the root node).
2) $\mathbb{P}$—Reward decay for nonoptimal moves.
3) $o$—Optimal move finding difficulty (expressed as a percentage of moves that are optimal in each state).
4) $\mathbb{B}$—Branching calculation.
5) $s_0$—Initial random number generator (RNG) seed.
6) $N$—Total leaves count.

### B. Local Parameters

1) $V$—Reward for the player when min–max Strategy is played.
2) $s_n$—RNG Seed for child nodes calculation.
3) $s_h$—RNG seed for the heuristic evaluation function simulation.
4) $n$—Leaves count
5) $d$—Tree depth.

The generator uses splittable pseudorandom number generators (Splittable PRNGs) [44], which are a special case of PRNGs where besides generating seemingly random numbers, an additional operation called *split* is defined. Split takes the current PRNG seed as an input and returns two seeds ($s_n$ and $s_h$) that will generate two seemingly independent pseudorandom sequences. Consequently, we set a generator seed $s_0$ for a root node of the game and then, when generating child nodes, each of them receives the respective individual seed, as a result of splitting the parent's seed. Based on this seed, all child node's properties are generated. Utilization of the other split-resulting seed ($s_h$) is described below in this section.

The heart of the generator is a procedure that generates a vector of successors from the current tree node. The successor

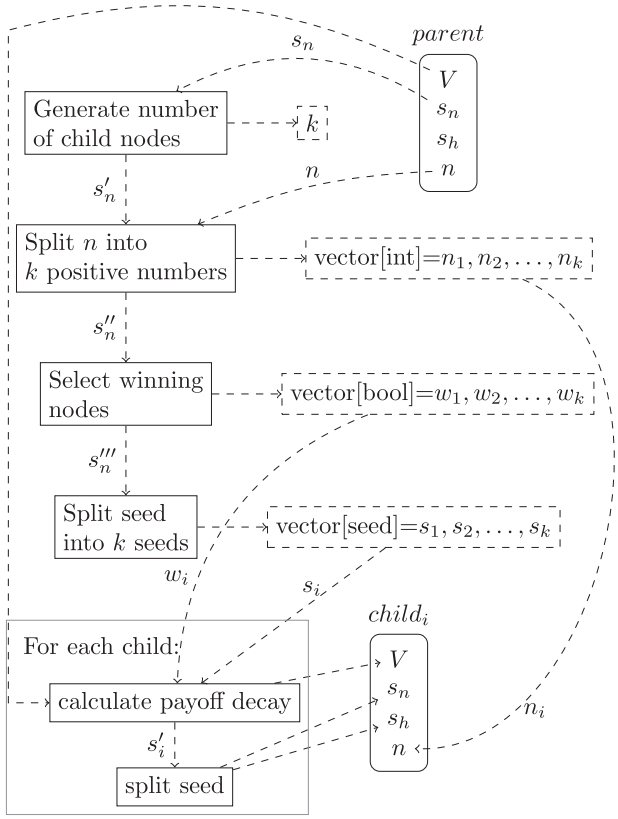


Fig. 2. Flowchart showing the procedure of successors generation. Solid border boxes depict subroutines and dashed border boxes describe the data. Arrows show the data flow.

generation procedure is summarized in Fig. 2 and works as follows: At first, PRNG with seed $s_n$ is used to choose a number of children for the current node. The exact procedure is a parameter of the method and is described in Section III-C. Let us denote by $k$ the number of child nodes of the current node, and by $n$ the number of all descendant nodes under the current node. First, $n$ is split into $k$ positive integers. The division is done with PRNG using a procedure that generates each feasible split with equal probability. Then, the resulting PRNG state is used to determine, which of the child nodes are designated to be the optimal solution and which are suboptimal ones. The number of optimal child nodes equals $o\%$, and all these nodes have the optimal reward $V$ assigned. Then the PRNG seed is split into $k$ seeds using Splittable PRNG capability and for each of $k$ child nodes using the new respective seed the following procedure is performed.

1) If the node is suboptimal, the payoff change is calculated using one of the procedures described in Section III-D and the resulting payoff is assigned as $V$ of the new node.
2) The resulting seed is split again into two seeds. The first resulting seed is assigned to $s_n$ of the child node, and the second one to $s_h$.

### C. Number of Child Nodes Calculation

To allow diverse shapes of the game tree, the generator provides a configurable procedure that decides how many child

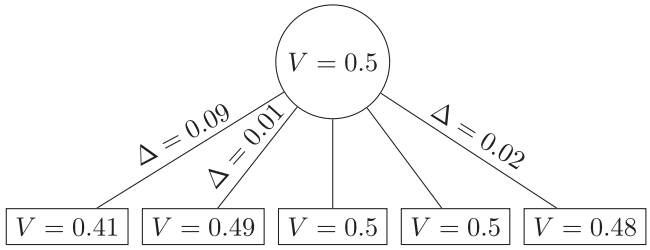


Fig. 3. Example of payoffs in the exponential payoff decay procedure (cf., Algorithm 1). Assume that $o = 40\%$, so 2 out of 5 moves are optimal. The figure presents the first player's node as a circle and the second player's nodes as rectangles. The $\Delta$ values drawn from the exponential distribution are presented next to the edges representing suboptimal moves.

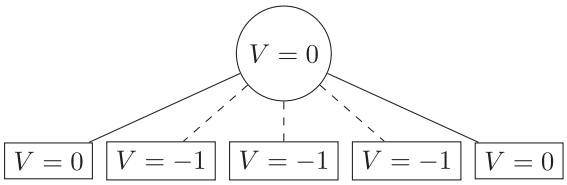


Fig. 4. Example of three point payoff decay. Assume that $o = 40\%$ so 2 out of 5 moves are optimal. The picture presents the first player's node as a circle and the second player's nodes as diamonds. Suboptimal moves are denoted with dashed edges and optimal ones with solid edges.

nodes a given node has. Basically, it works by drawing a number of children from a uniform random distribution within some range. For each node in a game, a different range can be defined by means of a linear interpolation of the range bounds.

The method is parameterized by a vector of triples: $(\mathrm{LB}, \mathrm{UB}, l)$, where LB is the lower bound of the range, UB is the upper bound of the range, and $l$ is the phase of the game. We define the game phase as a proportion of the number of leaves that do not lie in the subtree under the current node to the number of all leaves in the tree $\frac{N-q}{N} = 1 - \frac{q}{N}$, where $q$ is the number of nodes in the subtree under the current node. The root node has a game phase value of 1 and the leaves have values close to 0.

For each node the range for drawing the number of child nodes is calculated as follows:

1) calculate the game phase for the current node, denote is as $l^*$;
2) from the vector of interpolation points find the triple with the largest $l \leq l^*$, denote this triple as $(\underline{\mathrm{LB}}, \underline{\mathrm{UB}}, \underline{l})$. Similarly, find the triple with the smallest $l \geq l^*$, denote it as $(\overline{\mathrm{LB}}, \overline{\mathrm{UB}}, \overline{l})$;
3) then
   a) if both points are the same, then draw a value from the range $[\underline{\mathrm{LB}}, \underline{\mathrm{UB}}]$,
   b) if the first point does not exist, then draw a value from the range $[\overline{\mathrm{LB}}, \overline{\mathrm{UB}}]$,
   c) if the second point does not exist, then draw a value from the range $[\underline{\mathrm{LB}}, \underline{\mathrm{UB}}]$, and
   d) otherwise
      i) perform linear interpolation of the ranges based on the game phase

$$\mathrm{LB} \leftarrow \underline{\mathrm{LB}} \frac{l^* - \underline{l}}{\overline{l} - \underline{l}} + \overline{\mathrm{LB}} \left(1 - \frac{l^* - \underline{l}}{\overline{l} - \underline{l}}\right) \quad (2)$$

$$\mathrm{UB} \leftarrow \underline{\mathrm{UB}} \frac{l^* - \underline{l}}{\overline{l} - \underline{l}} + \overline{\mathrm{UB}} \left(1 - \frac{l^* - \underline{l}}{\overline{l} - \underline{l}}\right) \quad (3)$$

      ii) draw uniformly a value from $[\mathrm{LB}, \mathrm{UB}]$;
4) round the drawn number to the nearest integer.

The above approach allows managing the tree branching dynamics at various depths of the tree. Both the lower and upper bounds of the range from which the number of child nodes is drawn from are interpolated based on the game phase. This allows for modeling different game tree shapes, for instance with a high branching factor at the beginning of the game and its gradual decrease toward the end of the game.

**Algorithm 1:** Exponential Payoff Decay.

**Input:** $s, V$
**Output:** $V_{child}, s_i$
$(\Delta, s_i) \leftarrow DrawAndReturnSeed(exponential(\lambda), s)$
$V_{child} \leftarrow V - \Delta$

**Algorithm 2:** 3-Point Payoff.

**Input:** $s, V$
**Output:** $V_{child}, s_i$
$s_i \leftarrow s$
$V_{child} \leftarrow \max(V - 1, -1)$

### D. Payoff Decay Calculation

Within its current form the generator has two payoff decay calculators to choose from. One generates real-valued game payoffs and the other one works on a discrete set of payoffs $\{-1, 0, 1\}$. Both calculators take the parent's node value $V$ and PRNG seed $s_h$ as inputs.

Note that both calculators describe only the calculation of the game value for the player acting in the parent node. As the game is zero-sum, the other's player value changes, respectively.

*1) Exponential Payoff Decay:* The first calculator works by drawing a value $\Delta$ from an exponential distribution with parameter $\lambda$, and subtracting $\Delta$ from $V$ (see Algorithm 1). An example result is shown in Fig. 3.

*2) 3-Point Payoff:* The other calculator creates a game with three discrete payoff values: $-1$, 0, and 1. This calculator does not rely on PRNG and simply reduces the game value for the player by one. If the player's payoff is already equal to $-1$ it is not changed (see Algorithm 2). An example result is shown in Fig. 4.

## IV. Experimental Setup

In this section, we first provide a brief introduction to the MCTS algorithm (Section IV-A), which is an underlying method

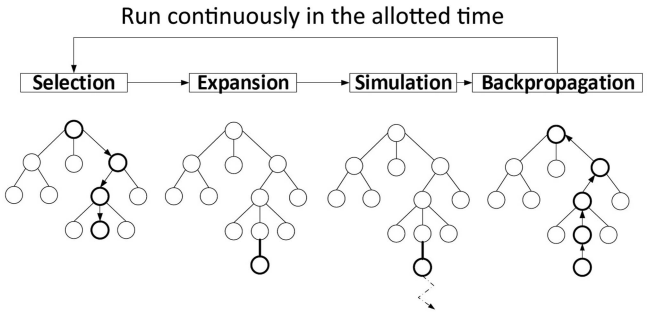


Fig. 5. Four phases of each MCTS iteration.

of most of the players tested in the experiments. Then, in Section IV-B, we present an idea of building strong game-agnostic evaluation function that could be incorporated into MCTS implementation. Finally, in Section IV-C, we briefly introduce the players (algorithms) used to test the proposed game-tree generator.

### A. Monte Carlo Tree Search

Let us start with a brief overview of the MCTS algorithm, commonly used as a game tree search method. For a detailed description of the algorithm, please consult one of the survey papers [11], [12]. The following description assumes a two-player constant-sum game, as this is a variant for which AutoGT is presented in this article.

MCTS is a simulation-based method that starts off with a single node (the root) and gradually builds an in-memory representation of a part of the game tree already visited in previous iterations. We will refer to this part (subtree) as the *MCTS tree*. In each node, MCTS gathers statistics, which in the standard version of the method consist of the average game outcome (result) of each player computed empirically in the iterations made so far that included this node, as well as the number of these iterations.

Each iteration consists of the following four phases, as depicted in Fig. 5.

1) *Selection*—The in-memory portion of the game tree is searched by making actions starting from the root node. The formula of choosing subsequent child nodes is called a *selection policy*. This phase ends with either making an action for which the next node is not yet stored in the MCTS tree or if a terminal node, i.e., without any children, is encountered. In the latter case, the MCTS algorithm skips directly to the backpropagation phase, with the actual game result obtained directly from the terminal game position.
2) *Expansion*—A new node (that represents the state reached after the last chosen action in the selection phase) is added to the tree. In some variants of MCTS implementations, more than one node can be added.
3) *Simulation*—This phase is also called the *Monte Carlo phase*. The idea is to sample the game result starting from the newly expanded node. Here, a full simulation of the game is performed in order to reach a terminal state. Players act according to the so-called *default policy*, which typically consists in making actions at random. When the simulation ends, the game outcome of the respective player (the one acting in the root node) is passed to backpropagation.
4) *Backpropagation*—The game outcome of the root node player is propagated back to all nodes visited in the selection phase. The statistics in those nodes are updated. Unless the number of iterations or time limit are up, the control flow goes back to the selection phase.

The most common selection policy is UCT [27]. In each node $s$, starting from the root node, an action $a^*$ (leading to a specific child node) is chosen according to the following:

$$a^* = \arg\max_{a \in A(s)} \left\{ Q(s,a) + C\sqrt{\frac{ln\,[N(s)]}{N(s,a)}} \right\} \tag{4}$$

where $A(s)$ is a set of actions available in state $s$, $Q(s,a)$ is the observed average result of playing action $a$ in state $s$, $N(s)$ is a number of times state $s$ has been visited in previous iterations, and $N(s,a)$—a number of times action $a$ has been sampled in state $s$ in previous iterations. $C$ controls the balance between exploration and exploitation and can be fine-tuned for a given game, however, quite a common choice is $C = \sqrt{2V_{\max}}$, where $V_{\max}$ is the highest possible result in the game. If two or more actions have the same value, the first or last action is typically chosen, depending on how the condition for checking the maximum value is implemented. In our implementation, we use the first-action approach. Other strategies include randomly selecting among the tied actions or utilizing additional heuristics.

The algorithm stops when a given stop condition is met. The two most popular ones are: 1) a number of iterations allotted for each decision an MCTS-based player makes in a game is reached or 2) a time limit per move is completed. When the stop condition is met, the MCTS-proposed action $a$, for the player that moves in the root node, is the one with the highest number of visits $N(\text{root}, a)$ or with the highest average score $Q(\text{root}, a)$. Both versions are popular in the literature. The latter one is used in the experiments presented in this article. Since actions with higher average scores are visited more frequently [due to the higher value of the $Q$ component in (4)], most of the times the same action maximizes both metrics.

### B. Heuristic Evaluation Function

Although the vanilla MCTS implementation does not require any game-specific heuristics, many of its extensions use them in some way, introducing heuristic bias with the aim of increasing the overall efficacy of the algorithm.

Typically, heuristic evaluation functions are either created manually by the game experts or trained based on past games using ML, such as the value function prediction model of AlphaGo [45]. In either case, the input to the evaluation function are the features of the current state. Based on these features, the heuristic indicates how good the state is for a particular player.

Since AutoGT does not generate any fictitious state representation, our idea to effectively implement the heuristic evaluation

function relies on utilizing the game-theory optimal score with a certain distraction. Let gto$(s)$ denote the game-theory optimal score for an active player in state $s$, i.e., its game score when both players play min-max strategy from state $s$ to the end of the game. The heuristic value $h(s)$ of state $s$ is expressed as follows:

$$h(s) = \text{gto}(s) + \psi X(s_h) \tag{5}$$

where $X(s_h)$ is a distortion from the range $[-d, d]$ calculated using the uniform real number generator initialized with seed $s_h$. $d$ is a difference between the highest evaluation value among all moves in the current state (gto$^{\#1}(s)$) and the next highest evaluation value in that state (gto$^{\#2}(s)$) smaller than gto$^{\#1}(s)$ (if all moves have the same value then $X(s_h) := 0$); $\psi$ is a scaling parameter.

### C. Tested Players

The following algorithms (players) have been considered in the evaluation of the efficacy and usefulness of the proposed AutoGT generator. In their description, $I$ denotes the number of simulations allotted for each MCTS decision step.

1) *Oracle*—Always makes the game-theory optimal action (according to the Nash equilibrium). Oracle is included as a reference method to establish the upper limit when assessing the other players.
2) *Random*—Randomly selects an action from the available options with a uniform probability. This method is included primarily to determine if certain scores in the current game configuration can be achieved by chance.
3) *MCTS-I*—A vanilla MCTS algorithm described in Section IV-A.
4) *MCTS-$\epsilon$-Greedy-I*—Uses the heuristic [see (5)] in the simulation phase of MCTS to modify the default policy. With probability $\epsilon$, it plays the action that maximizes this heuristic. Otherwise, it selects a random action with uniform probability like the Random player. This method is a very popular default policy and has been reported in numerous studies [46], [47]. In the experiments, $\epsilon = 0.5$ is used.
5) *MCTS-Softmax-$\beta$-I*—Relies on another popular modification of the default policy [48]. Instead of selecting actions with a uniform probability, they are chosen according to the Boltzmann distribution, a.k.a. softmax

$$P(a) = \frac{e^{\beta h(a)}}{\sum_{a' \in A} e^{\beta h(a')}} \tag{6}$$

where $h(a)$ denotes the heuristic evaluation of the state that action $a$ leads to (it is unequivocal since the games are deterministic with alternate moves); $\beta$ is the scaling parameter. High-value actions are more likely to be chosen than low-value ones, though the latter still have a chance to be selected.
6) *MCTS-p-CutOff-I*—Is based on the MCTS modification known as *early cut-off* or *early termination* [12], [49]. It modifies the default policy by introducing a probability parameter $p$ that determines, at each step of the simulation, whether this simulation should terminate in the current node. In such a case, the heuristic evaluation of the last visited state is returned as the game simulation outcome. Otherwise, the algorithm selects an action with uniform probability. The main advantage of introducing such a heuristic bias into MCTS is the computational gain stemming from shortening the simulations.

## V. Results

To demonstrate the potential of the proposed generator, we conducted experiments involving 48 AutoGT parameterizations and 29 players implementing variants of the six generic algorithms listed in the previous section.

Global generator parameterizations differed in the following four parameters.

1) $N \in \{10^7, 10^8, 10^9, 10^{10}\}$.
2) $o \in \{0.1, 0.25, 0.4\}$.
3) $\mathbb{P}$: 1) exponential payoff decay with $\lambda = 5$ (denoted as $E$ in the results) and 2) 3-Point payoff (denoted as 3).
4) $\mathbb{B}$: 1) a vector $[(2, 4, 0), (3, 10, 0.5), (2, 4, 1)]$ for both players (denoted as A in the results) and 2) the above vector for the first player and a vector $[(3, 10, 0), (2, 4, 0.5), (3, 10, 0)]$ for the second player (denoted as B in the results).

A Cartesian product of those four parameters was used, so all together 48 different parameter settings were tested in the experiments. In all cases, $V^*$ was set to 0. For each parameter combination, 50 game instances with different $s_0$ values were used. This gives a total of 2400 distinct games.

In a round-robin tournament each player was pitted against each other player in duels composed of 200 games, in half of them in the role of the first player and in the other half as the second player. We used a score measure $S_p = (W + \frac{D}{2})/T$, where $p$ is a player, $W$ is a number of matches the player won, $D$ is a number of player's draws and $T$ is a total number of matches played by the player. $S_p$ was used to check how various generator parameterizations differentiate players' outcomes. For this purpose, we defined an aggregated measure called DScore (denoted by DS)

$$\text{DS} = \sum_{p, q \neq p} |S_p - S_q| / (P^2 - P) \tag{7}$$

where $p$ and $q$ are players, and $P$ is the number of players. DS is the average absolute pairwise difference in players' scores. Table I presents the DS values (for 50 different seeds $s_0$ per each parameterization) for all players considered in the experiment.

It can be seen in Table I that the size of the game tree (expressed as a number of leaf nodes) is an important factor in differentiating players' scores. Small games ($10^7$ and $10^8$ leaves) yield smaller DS values than large games ($10^9$ and $10^{10}$ nodes). The most differentiating games have $\mathbb{P}$ set to $E$ (exponential payoff decay).

Table II presents, in the descending order, the $S_p$ values of all players considered in the experiment. It can be seen from the table that games generated by AutoGT well differentiate the players. As expected, Oracle achieves the highest score with securing only wins and draws. Excluding the two baseline

TABLE I
DS VALUES PER GAME TYPE, AVERAGED OVER 50 DIFFERENT SEEDS $s_0$

| Game | $DS$ | | |
|---|---|---|---|
| $N10^9$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.4$ | 0.202 | $N10^8$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.4$ | 0.156 |
| $N10^9$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.1$ | 0.202 | $N10^7$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.4$ | 0.154 |
| $N10^9$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.25$ | 0.201 | $N10^{10}$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.1$ | 0.154 |
| $N10^{10}$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.1$ | 0.181 | $N10^8$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.25$ | 0.153 |
| $N10^{10}$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.25$ | 0.181 | $N10^8$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.4$ | 0.141 |
| $N10^9$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.25$ | 0.179 | $N10^7$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.4$ | 0.139 |
| $N10^{10}$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.4$ | 0.178 | $N10^{10}$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.1$ | 0.129 |
| $N10^8$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.1$ | 0.178 | $N10^8$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.25$ | 0.126 |
| $N10^8$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.25$ | 0.177 | $N10^8$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.1$ | 0.126 |
| $N10^{10}$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.25$ | 0.172 | $N10^9$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.1$ | 0.123 |
| $N10^{10}$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.1$ | 0.172 | $N10^8$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.4$ | 0.121 |
| $N10^{10}$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.4$ | 0.171 | $N10^{10}$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.25$ | 0.121 |
| $N10^9$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.1$ | 0.170 | $N10^9$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.25$ | 0.113 |
| $N10^8$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.1$ | 0.169 | $N10^{10}$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.4$ | 0.108 |
| $N10^7$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.1$ | 0.168 | $N10^7$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.1$ | 0.100 |
| $N10^{10}$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.4$ | 0.167 | $N10^7$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.25$ | 0.099 |
| $N10^7$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.1$ | 0.167 | $N10^8$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.1$ | 0.098 |
| $N10^10$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.25$ | 0.167 | $N10^9$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.4$ | 0.096 |
| $N10^7$-$\mathbb{B}A$-$\mathbb{P}E$-$o0.25$ | 0.162 | $N10^7$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.4$ | 0.092 |
| $N10^9$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.4$ | 0.162 | $N10^8$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.25$ | 0.089 |
| $N10^9$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.1$ | 0.162 | $N10^7$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.1$ | 0.079 |
| $N10^9$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.25$ | 0.161 | $N10^7$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.25$ | 0.073 |
| $N10^7$-$\mathbb{B}B$-$\mathbb{P}E$-$o0.25$ | 0.160 | $N10^8$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.4$ | 0.070 |
| $N10^9$-$\mathbb{B}A$-$\mathbb{P}3$-$o0.4$ | 0.159 | $N10^7$-$\mathbb{B}B$-$\mathbb{P}3$-$o0.4$ | 0.056 |

TABLE II
$S_p$ VALUES OF ALL PLAYERS CONSIDERED IN THE EXPERIMENTS

| Player | $S_p$ | 0.95 CI | Symbol |
|---|---|---|---|
| Oracle | 0.732 | 0.0006 | O |
| MCTS-0.5-Greedy-10000-H0 | 0.606 | 0.0011 | G-0 |
| MCTS-0.5-Greedy-10000-H1 | 0.603 | 0.0011 | G-1 |
| MCTS-Softmax-1-10000-H1 | 0.602 | 0.0011 | S-1 |
| MCTS-0.5-Greedy-10000H3 | 0.601 | 0.0011 | G-3 |
| MCTS-Softmax-20-10000H3 | 0.600 | 0.0011 | S20 |
| MCTS-Softmax-1-10000H3 | 0.599 | 0.0011 | S-3 |
| MCTS-cutting-10000-H0 | 0.596 | 0.0011 | |
| MCTS-cutting-10000-H1 | 0.596 | 0.0011 | |
| MCTS-10000 | 0.595 | 0.0011 | M |
| MCTS-cutting-10000H3 | 0.592 | 0.0011 | |
| MCTS-Softmax-20-10000-H0 | 0.592 | 0.0011 | |
| MCTS-Softmax-1-10000-H0 | 0.591 | 0.0011 | |
| MCTS-Softmax-20-10000-H1 | 0.586 | 0.0011 | |
| MCTS-0.5-Greedy-1000-H0 | 0.469 | 0.0013 | |
| MCTS-Softmax-20-1000-H0 | 0.458 | 0.0012 | |
| MCTS-Softmax-1-1000-H0 | 0.457 | 0.0012 | |
| MCTS-0.5-Greedy-1000-H1 | 0.456 | 0.0013 | |
| MCTS-Softmax-1-1000-H1 | 0.452 | 0.0013 | |
| MCTS-cutting-1000-H0 | 0.448 | 0.0013 | |
| MCTS-cutting-1000-H1 | 0.446 | 0.0013 | |
| MCTS-Softmax-20-1000H3 | 0.442 | 0.0013 | |
| MCTS-0.5-Greedy-1000H3 | 0.442 | 0.0013 | |
| MCTS1000 | 0.440 | 0.0013 | |
| MCTS-Softmax-1-1000H3 | 0.434 | 0.0013 | |
| MCTS-cutting-1000H3 | 0.429 | 0.0013 | |
| MCTS-Softmax-20-1000-H1 | 0.400 | 0.0013 | |
| MCTS-100 | 0.229 | 0.0013 | |
| Uniform | 0.004 | 0.0013 | |

Names of players that applied the heuristic function (eq. 5) have the Suffix $H\psi$, where $\psi$ is the heuristic parameter. We have included the lengths of 95% confidence intervals, computed using the t-distribution, over the population of all scores $S_p$ achieved by the players.

TABLE III
$S_p$ RESULTS OF THE BEST PLAYERS FOR GAME $N10^9$-$\mathbb{P}3$-$\mathbb{B}B$-$o0.1$

| | O | G-0 | G-1 | S-1 | G-3 | S20 | S-3 | M |
|---|---|---|---|---|---|---|---|---|
| O | 0.500 | 0.734 | 0.700 | 0.723 | 0.694 | 0.670 | 0.698 | 0.682 |
| G-0 | 0.354 | 0.620 | 0.611 | 0.633 | 0.615 | 0.593 | 0.634 | 0.608 |
| G-1 | 0.338 | 0.600 | 0.608 | 0.624 | 0.605 | 0.568 | 0.632 | 0.589 |
| S-1 | 0.333 | 0.593 | 0.585 | 0.596 | 0.583 | 0.564 | 0.628 | 0.572 |
| G-3 | 0.343 | 0.592 | 0.593 | 0.611 | 0.602 | 0.568 | 0.632 | 0.589 |
| S20 | 0.325 | 0.583 | 0.576 | 0.566 | 0.573 | 0.579 | 0.604 | 0.565 |
| S-3 | 0.325 | 0.582 | 0.579 | 0.591 | 0.596 | 0.574 | 0.613 | 0.579 |
| M | 0.333 | 0.600 | 0.600 | 0.608 | 0.601 | 0.573 | 0.626 | 0.580 |

players (Oracle and Uniform) the difference in $S_p$ between the best and the worst player equals 0.377. The most important factor influencing MCTS-based players performance is the iteration budget. $S_p = 0.586$ of the worst player with 10 000 iterations budget is higher by 0.117 than the best 1000-iteration-budget MCTS player. When looking separately at 1000 and 10 000 iteration budget players, it can be observed that in both cases, the best performing player is the 0.5-Greedy variant with $H0$ ($\psi = 0$ in (5), which means the exact state evaluation). The worst player is the same in both cases: Softmax-20 with $H1$ heuristic variant [$\psi = 1$ in (5)]. A particular heuristic bias in MCTS seems to have a lower impact with higher number of iterations, which aligns with the established understanding of the algorithm. The differences between the top and worst-performing players in the 1000 and 10 000 iteration groups equal 0.02 and 0.069, respectively.

Due to space limitations, we will present more detailed results only for the top-performing players and MCTS-10000 regarded as a baseline. The players chosen for further comparison are distinguished by an nonempty Symbol value in the third column of Table II.

Tables III–V present detailed results for the eight selected players in three manually selected games. Each table presents $S_p$ values averaged across 50 seeds. Players in rows were the first (starting) players in a game and the column players were the second (responding) ones.

Generally speaking, the results show that particular game parameterizations are more favorable for particular algorithms. While the O player is, by definition, the top-1 in all comparisons, the remaining players achieve different relative results in all three setups. For instance, the baseline M player, presumably the weakest in this group, performs better than S-3 and S20 in game 1e+09$-\mathbb{P}3$-$\mathbb{B}B$-$o0.1$ (see Table III). Also, the order of players other that O varies for particular game/generator settings, e.g., G-3 fares better than S-1in Table III and quite the opposite in Table IV. A similar observation applies, for instance, to algorithms S-1 and G-1in Tables IV and V. We also computed 95% confidence intervals using the t-distribution. With a precision of three decimal places, only three unique values appear: $\pm 0.000$ in the O versus O matchup, $\pm 0.007$ in other games involving the O player, and $\pm 0.011$ for the remaining games.

TABLE IV
$S_p$ RESULTS OF THE BEST PLAYERS FOR GAME $N10^9$-$\mathbb{P}E$-$\mathbb{B}A$-$o0.4$

| | O | G-0 | G-1 | S-1 | G-3 | S20 | S-3 | M |
|---|---|---|---|---|---|---|---|---|
| O | 0.500 | 0.742 | 0.747 | 0.716 | 0.743 | 0.743 | 0.730 | 0.755 |
| G-0 | 0.236 | 0.547 | 0.549 | 0.484 | 0.549 | 0.514 | 0.504 | 0.550 |
| G-1 | 0.235 | 0.542 | 0.544 | 0.480 | 0.542 | 0.512 | 0.496 | 0.543 |
| S-1 | 0.244 | 0.556 | 0.557 | 0.512 | 0.556 | 0.536 | 0.522 | 0.555 |
| G-3 | 0.233 | 0.541 | 0.546 | 0.477 | 0.544 | 0.508 | 0.497 | 0.543 |
| S20 | 0.236 | 0.547 | 0.543 | 0.501 | 0.548 | 0.520 | 0.509 | 0.552 |
| S-3 | 0.234 | 0.547 | 0.552 | 0.506 | 0.559 | 0.521 | 0.515 | 0.554 |
| M | 0.234 | 0.543 | 0.549 | 0.483 | 0.551 | 0.507 | 0.507 | 0.545 |

TABLE V
$S_p$ RESULTS OF THE BEST PLAYERS FOR GAME $N10^{10}$-$\mathbb{P}3$-$\mathbb{B}B$-$o0.25$

| | O | G-0 | G-1 | S-1 | G-3 | S20 | S-3 | M |
|---|---|---|---|---|---|---|---|---|
| O | 0.500 | 0.649 | 0.646 | 0.688 | 0.621 | 0.595 | 0.604 | 0.586 |
| G-0 | 0.399 | 0.558 | 0.570 | 0.612 | 0.558 | 0.534 | 0.544 | 0.531 |
| G-1 | 0.372 | 0.541 | 0.545 | 0.599 | 0.540 | 0.521 | 0.522 | 0.508 |
| S-1 | 0.397 | 0.514 | 0.532 | 0.555 | 0.530 | 0.512 | 0.525 | 0.503 |
| G-3 | 0.356 | 0.518 | 0.536 | 0.591 | 0.523 | 0.505 | 0.519 | 0.496 |
| S20 | 0.309 | 0.482 | 0.502 | 0.562 | 0.492 | 0.495 | 0.491 | 0.467 |
| S-3 | 0.363 | 0.537 | 0.556 | 0.599 | 0.544 | 0.515 | 0.521 | 0.509 |
| M | 0.337 | 0.506 | 0.516 | 0.569 | 0.507 | 0.485 | 0.502 | 0.480 |

### A. Time Performance

To present the time performance of AutoGT, we measured the computation time of generating possible actions from nodes, as well as the time of generating the successor after playing the action. All performance measurements were done on Intel Xeon Silver 4116 CPU at 2.10 GHz with Devuan GNU/Linux, running 40 processes in parallel.

The measuring program traversed a whole game tree in the depth-first manner. The program was run for each of the 2400 games used in the experiments and was able to traverse 1 752 023 nodes per second on average.

In the second experiment, we ran a match between two copies of MCTS-1000 on a subset of games with CPU profiler attached and measured the time spent on calculating the game related aspects (a list of possible actions, a successor of the state) versus the whole time spent on running the match. The average time spent in game-related procedures took 48% of the overall match time.

It is hard to compare the above time scores with other generators due to the lack of similar works. Perhaps a more meaningful indication of the practical feasibility of AutoGT will be expressing the generation time in seconds. In the first task, generating a game with $N = 10^7$ ($10^{10}$) nodes takes on average 8.94 (8967.23) s.

### B. Tic–Tac–Toe (TTT) Recreation Experiment

To validate our approach, we conducted experiments using the solved game of *Tic–Tac–Toe* (TTT), comparing the real game tree with our approximated game tree generated by AutoGT. In the regular TTT setting, the MCTS agent required eight iterations to confidently reach an average score above 0.9 against a Random agent, while AutoGT achieved the same confidence level in seven iterations. When MCTS competed against GTO, the regular game needed 65 iterations to reach an average score above 0.49, whereas AutoGT required 58 iterations.

However, when the roles were switched, the numbers of required iterations differed significantly. In the Random versus MCTS scenario, the regular game required 75 iterations, while AutoGT needed only 49. Similarly, for GTO versus MCTS, the regular game demanded 640 iterations, compared to 410 for AutoGT. We observed that, to reduce this difference, the only requirement is to change the *Optimal Move Finding Difficulty* parameter to be player dependent.

The average score in nodes was 0.648 for the regular game and 0.57 for AutoGT, with average leaf scores of 0.604 and 0.55, respectively. Both had a GTO score of 0.5. The node and leaf counts were also comparable, with the regular game having 549 946 nodes and 255 168 leaves, while AutoGT had 663 962 nodes and the same number of leaves. The depth distribution of leaf nodes for the regular game, the distribution across depths $[10, 9, 8, 7, 6]$ was $[127872, 72576, 47952, 5328, 1440]$. In comparison, AutoGT showed a distribution of $[108367, 42480, 64845, 32392, 7084]$ for the same depths.

### C. Examples of Parameter Configurations for Benchmarking

In this section, we discuss selected AutoGT parameter settings and their impact on the evaluation of tree-search algorithms. The following setups may serve as inspirations for further experiments.

1) *Low $o$ and low $\mathbb{B}$:* This setting represents a low percentage of optimal moves among children, combined with a low branching factor. It generates trees in which only one move is likely to be optimal. Specifically, when $\mathbb{B}$ is small in the early phases of the game, there is a high chance that decision nodes closer to the root will split the tree into optimal and suboptimal subtrees. If a player misses the optimal action in such a tree, they may never have a chance to recover.
2) *Low $o$ with high versus low $N$:* With other parameters fixed, increasing $N$ leads to deeper trees and therefore more decision points. Algorithms that perform well for low values of $N$ but struggle with higher ones may be less stable (repeatable).
3) *Various distortions of heuristic [$\psi X(s_h)$ in (5)]:* By modifying the heuristic function through controlled distortions, one can simulate different levels of inaccuracy in heuristic evaluations. This allows to analyze how robust various search algorithms are to imperfect heuristic guidance.
4) *Steep versus smooth payoff decays:* Steep payoff decay is discrete (e.g., [100, −100]), whereas smooth decay follows a continuous function. On the one hand, smooth decay may help algorithms recover from suboptimal moves by providing a gradual reward transition. On the other hand, small differences between payoffs can make it harder for algorithms to distinguish optimal actions, potentially degrading their performance. Comparing how

different algorithms handle these two cases provides information on their sensitivity to reward structures.

5) *Many parameter combinations versus various root node RNG seeds:* By varying the root node RNG seed ($s_0$), while keeping other parameters fixed, one can analyze which algorithms are generally more robust to randomness. In addition, testing many parameter combinations allows to determine the factors that correlate most with stability in a given algorithm.
6) *Fixed $N$ and $o$ with high $\mathbb{B}$ versus low $\mathbb{B}$:* When the number of leaves ($N$) is fixed, choosing drastically different values of the branching factor ($\mathbb{B}$) results in trees of significantly different depths. This setup evaluates the algorithm's ability to either select the best action from many choices in a shallow tree or navigate a long sequence of decisions with fewer choices at each step. Analyzing algorithm's performance in both scenarios provides insight into its adaptability to different tree structures.
7) *Low $N$ and a variety of other parameters:* Since AutoGT stores the optimal Min–Max value in each node, it enables a direct evaluation of the algorithms designed to solve games, such as retrograde analysis, exact solvers, or counterfactual regret minimization. By comparing the algorithm's computed values with the ground truth stored in the generator, one can measure solution accuracy, convergence speed, and error rates. Low values of $N$ are recommended to ensure that experiments with various parameter settings remain computationally feasible.

## VI. Conclusion

Tree search algorithms represent a pivotal area of research in game domain. While many game-related approaches concentrate on specific games—either by solving them or by creating the strongest possible computer player—we strongly believe that the field could benefit from evaluations performed across a vast array of games.

In this article, we introduce AutoGT—a robust game tree generation algorithm designed for the empirical evaluation of various tree-search algorithms.

Our proposed method employs a top–down propagation strategy. Starting from the root node, several parameters, such as the expected reward in the root, random number generator seed, and the difficulty of finding the optimal move, are propagated according to the predefined rules down the tree. Some parameters are public, while others should be used solely for the generator's purpose and remain hidden from the tree-search algorithms. To generate a subtree of a given node, only the information stored in that node is required. When testing tree-search algorithms, this information should be cached in the tree. The generator can then be dynamically queried to produce the next portion of the tree. The state of the generator can be accurately set by utilizing a Splittable PRNG.

One of the main advantages of the algorithm is its ability to operate in a top–down fashion, allowing for efficient handling of trees of arbitrary depth, while maintaining a game-theory optimal score at each node. Typically, this condition would necessitate solving the entire tree upfront. Our generator successfully combines the benefits of both above aspects due to its specific propagation procedure.

The proposed method incurs minimal computational overhead, rendering it highly suitable for large-scale experiments that compare tree-search algorithms. Its full source code has been released under the GPL license. For more details, please refer to the Appendix.

It is important to note that AutoGT is not intended to replace real-world benchmarks, but rather to complement them by providing a controlled environment for systematically exploring algorithm behavior. At the same time, by extending the range of testable conditions, AutoGT offers a platform to explore how algorithms generalize beyond domain-specific benchmarks.

Future work will focus on extending the generator to: 1) nonzero-sum games; 2) games with an arbitrary number of players; and 3) incomplete information and nondeterministic games. The AutoGT architecture makes the first two of the above extensions straightforward. The nonzero-sum property requires keeping separate $V_1, V_2$ parameters instead of a single $V$ for the players' best results and modifying the payoff decay accordingly. Arbitrary number of players requires first the above extension to be implemented, and then a payoff for each player will require a separate $V_i$.

Extending the generator to accommodate games with incomplete information is more challenging. It may be possible to treat incomplete information as a form of nondeterminism, in that actions performed without hidden information incorporate an element of uncertainty or risk, similar to random events. An alternative method to address incomplete information involves introducing an artificial state representation within nodes. This strategy allows the generator to randomly assign subsets of this representation to each player, treating it as the information currently accessible to that player. Moreover, it could potentially extend the AutoGT applicability from testing pure tree-search algorithms to hybrid methods utilizing heuristics for node assessment.

## Appendix
## Code Availability

The source code of the generator and the programs used in experiments is available on Gitlab: https://gitlab.com/jan-karwowski/game-tree-generator-pub. The repository contains README.md file that explains the usage of programs.

The generator provides a convenient function `run_with_parsed_game`, which makes it easy to use AutoGT without knowing its internals.

## References

[1] J. V. Neumann and O. Morgenstern, *Theory of Games and Economic Behavior, 2nd Rev*. Princeton, NJ, USA: Princeton Univ. Press, 1947.
[2] D. Michie, "Game-playing and game-learning automata," in *Proc. Adv. Program. Non-Numerical Comput.*, 1966, pp. 183–200.
[3] A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM J. Res. Dev.*, vol. 3, no. 3, pp. 210–229, 1959.
[4] J. Mańdziuk, *Knowledge-Free and Learning-Based Methods in Intelligent Game Playing, Ser. Studies in Computational Intelligence*. New York, NY, USA: Springer, 2010.

[5] J. McCarthy, "Chess as the drosophila of AI," in *Proc. Comput., Chess, Cognit.*, 1990, pp. 227–237.

[6] S. Gelly et al., "The grand challenge of computer Go: Monte Carlo tree search and extensions," *Commun. ACM*, vol. 55, no. 3, pp. 106–113, Mar. 2012.

[7] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2016, pp. 2094–2100. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/10295

[8] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.

[9] C. OpenAI et al., "Dota 2 with large scale deep reinforcement learning," 2019, *arXiv:1912.06680*.

[10] D. M. Kreps, "Nash equilibrium," in *Proc. Game Theory*, 1989, pp. 167–177.

[11] M. Świechowski, K. Godlewski, B. Sawicki, and J. Mańdziuk, "Monte Carlo tree search: A review of recent modifications and applications," *Artif. Intell. Rev.*, vol. 56, pp. 2497–2562, 2023, doi: 10.1007/s10462-022-10228-y.

[12] C. B. Browne et al., "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, Mar. 2012.

[13] B. Pell, "A strategic metagame player for general chess-like games," *Comput. Intell.*, vol. 12, no. 1, pp. 177–198, 1996.

[14] M. Genesereth, N. Love, and B. Pell, "General game playing: Overview of the AAAI competition," *AI Mag.*, vol. 26, no. 2, pp. 62–62, 2005.

[15] M. Świechowski, H.-S. Park, J. Mańdziuk, and K.-J. Kim, "Recent advances in general game playing," *Sci. World J.*, vol. 2015, 2015, Art. no. 986262.

[16] D. Perez-Liebana et al., "The 2014 general video game playing competition," *IEEE Trans. Comput. Intell. AI Games*, vol. 8, no. 3, pp. 229–243, Sep. 2016.

[17] R. D. Gaina, S. Devlin, S. M. Lucas, and D. Perez-Liebana, "Rolling horizon evolutionary algorithms for general video game playing," *IEEE Trans. Games*, vol. 14, no. 2, pp. 232–242, Jun. 2022.

[18] T. S. Nielsen, G. A. B. Barros, J. Togelius, and M. J. Nelson, "Towards generating arcade game rules with VGDL," in *Proc. IEEE Conf. Comput. Intell. Games*, 2015, pp. 185–192.

[19] C. Browne and F. Maire, "Evolutionary game design," *IEEE Trans. Comput. Intell. AI Games*, vol. 2, no. 1, pp. 1–16, Mar. 2010.

[20] V. Hom and J. Marks, "Automatic design of balanced board games," in *Proc. AAAI Conf. Artif. Intell. Interactive Digit. Entertainment*, 2007, vol. 3, pp. 25–30.

[21] A. M. Smith and M. Mateas, "Variations forever: Flexibly generating rulesets from a sculptable design space of mini-games," in *Proc. 2010 IEEE Conf. Comput. Intell. Games*, 2010, pp. 273–280.

[22] M. Hendrikx, S. Meijer, J. Van Der Velden, and A. Iosup, "Procedural content generation for games: A survey," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 9, no. 1, pp. 1–22, 2013.

[23] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, "Search-based procedural content generation: A taxonomy and survey," *IEEE Trans. Comput. Intell. AI Games*, vol. 3, no. 3, pp. 172–186, Sep. 2011.

[24] A. Taitler et al., "The 2023 International Planning Competition," *AI Mag.*, vol. 45, no. 2, pp. 280–296, 2024. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/aaai.12169

[25] F. V. Ameneyro and E. Galván, "Towards understanding the effects of evolving the MCTS UCT selection policy," in *Proc. IEEE Symp. Ser. Comput. Intell.*, 2022, pp. 1683–1690.

[26] E. Galván and F. V. Ameneyro, "An analysis on the effects of evolving the Monte Carlo tree search upper confidence for trees selection policy on unimodal, multimodal and deceptive landscapes," *Inf. Sci.*, vol. 715, 2025, Art. no. 122226, [Online]. Available:https://www.sciencedirect.com/science/article/pii/S0020025525003585

[27] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. 17th Eur. Conf. Mach. Learn.*, 2006, pp. 282–293.

[28] J. Mańdziuk and P. Walczak, "Monte Carlo tree search with metaheuristics," in *Proc. Artif. Intell. Soft Comput*, 2023, pp. 134–144.

[29] F. Frydenberg, K. R. Andersen, S. Risi, and J. Togelius, "Investigating MCTS modifications in general video game playing," in *Proc. IEEE Conf. Comput. Intell. Games*, 2015, pp. 107–113.

[30] D. J. N. J. Soemers, C. F. Sironi, T. Schuster, and M. H. M. Winands, "Enhancements for real-time Monte-Carlo tree search in general video game playing," in *Proc. IEEE Conf. Comput. Intell. Games*, 2016, pp. 1–8.

[31] K. Waldzik and J. Mańdziuk, "An automatically generated evaluation function in general game playing," *IEEE Trans. Comput. Intell. AI Games*, vol. 6, no. 3, pp. 258–270, Sep. 2014.

[32] M. Świechowski and J. Mańdziuk, "Self-adaptation of playing strategies in general game playing," *IEEE Trans. Comput. Intell. AI Games*, vol. 6, no. 4, pp. 367–381, Dec. 2014.

[33] M. Świechowski, J. Mańdziuk, and Y. S. Ong, "Specialization of a UCT-Based general game playing program to single-player games," *IEEE Trans. Comput. Intell. AI Games*, vol. 8, no. 3, pp. 218–228, Sep. 2016.

[34] D. Silver et al., "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[35] L. Tommy, M. Hardjianto, and N. Agani, "The analysis of alpha beta pruning and MTD (f) algorithm to determine the best algorithm to be implemented at connect four prototype," in *Proc. IOP Conf. Series: Mater. Sci. Eng.*, 2017, Art. no. 012044.

[36] I. Musah, D. K. Boah, and B. Seidu, "A comprehensive review of solution methods and techniques for solving games in game theory," *J. Game Theory*, vol. 9, no. 2, pp. 25–31, 2020.

[37] M. Małkiński, S. Pawlonka, and J. Mańdziuk, "Reasoning limitations of multimodal large language models a case study of Bongard Problems," in *Proc Int. Conf. Mach. Learn.*, May 2025.

[38] Y. Chang et al., "A survey on evaluation of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 3, pp. 1–45, Mar. 2024, doi: 10.1145/3641289.

[39] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman, "Leveraging procedural generation to benchmark reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 2048–2056.

[40] C. Ansotegui and E. Torres, "A benchmark generator for combinatorial testing," in *Proc IEEE Trans. Softw. Eng.*, Sep. 2022.

[41] A. Torralba, J. Seipp, and S. Sievers, "Automatic instance generation for classical planning," in *Proc. Int. Conf. Automated Plan. Scheduling*, 2021, pp. 376–384.

[42] A. Younes, P. Calamai, and O. Basir, "Generalized benchmark generation for dynamic combinatorial problems," in *Proc. 7th Annu. Workshop Genet. Evol. Comput. (GECCO '05)*New York, NY, USA: Association for Computing Machinery, 2005, pp. 25–31, [Online]. Available:https://doi.org/10.1145/1102256.1102262

[43] D. Yazdani, M. N. Omidvar, D. Yazdani, K. Deb, and A. H. Gandomi, "GNBG: A generalized and configurable benchmark generator for continuous numerical optimization," 2023, *arXiv:2312.07083*.

[44] G. L. Steele Jr., D. Lea, and C. H. Flood, "Fast splittable pseudorandom number generators," in *Proc. ACM Int. Conf. Object Oriented Program. Syst. Lang. Appl.*, 2014, pp. 453–472, doi: 10.1145/2660193.2660195.

[45] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[46] C. Dann et al., "Guarantees for Epsilon-Greedy reinforcement learning with function approximation," in *Proc. 39th Int. Conf. Mach. Learn.*, 2022, vol. 162, pp. 4666–4689. [Online]. Available: https://proceedings.mlr.press/v162/dann22a.html

[47] M. Wunder, M. L. Littman, and M. Babes, "Classes of multiagent Q-learning dynamics with Epsilon-Greedy exploration," in *Proc. 27th Int. Conf. Mach. Learn.*, 2010, pp. 1167–1174.

[48] H. Finnsson and Y. Björnsson, "CadiaPlayer: A simulation-based general game player," *IEEE Trans. Comput. Intell. AI Games*, vol. 1, no. 1, pp. 4–15, Mar. 2009.

[49] R. Lorentz, "Early playout termination in MCTS," in *Proc. Adv. Comput. Games*, 2015, pp. 12–19.